

汇编语言程序设计

4.2 分支程序结构

汇编语言程序设计

4.2.1 无条件转移指令

指令寻址方式

JMP指令

寻址方式 (Addressing)

➤ 通过地址访问数据或指令

➤ 数据寻址: **操作数在哪儿呢?**

指令执行过程中,
访问所需要操作的数据(操作数)

00405000H

➤ 指令寻址:

一条指令执行后, 确定执行
的下一条指令的位置

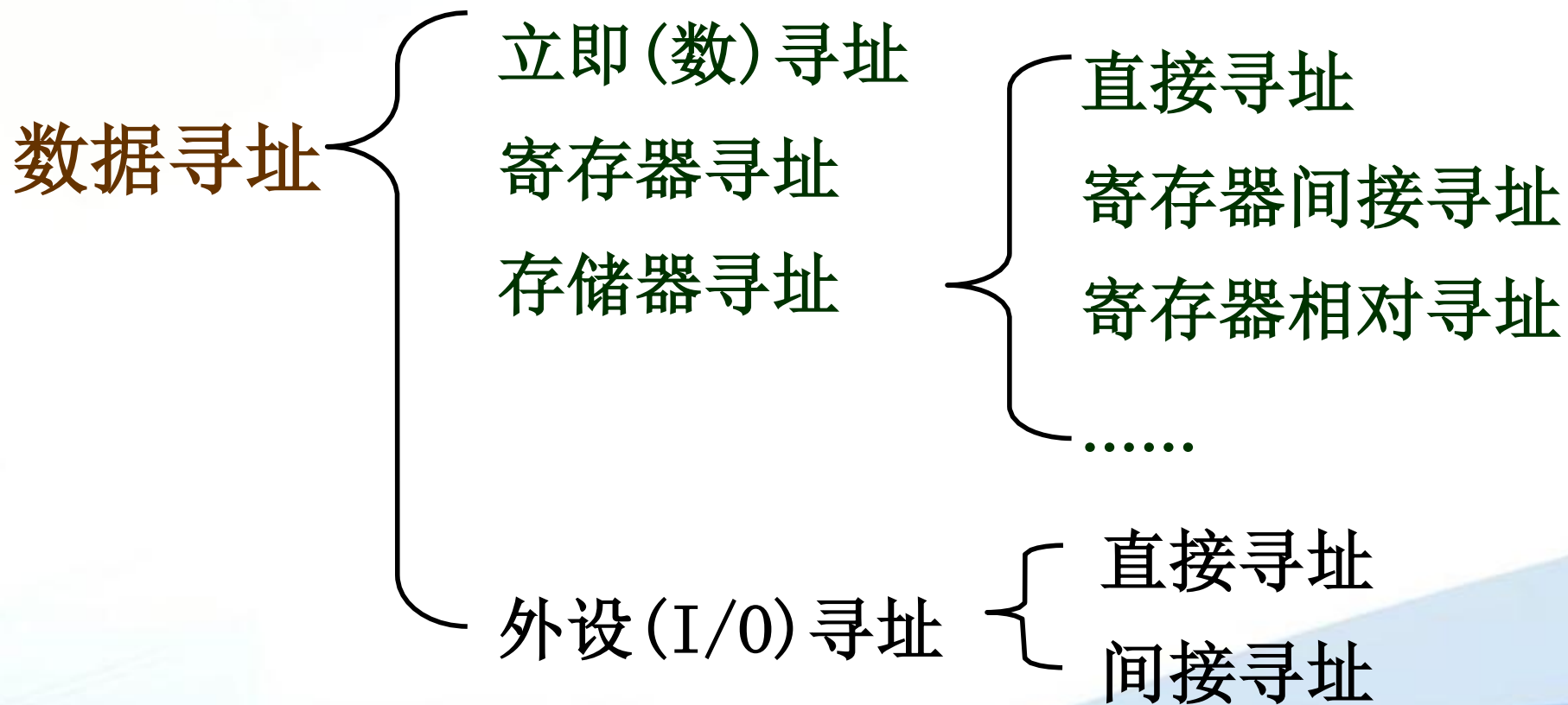
00401000H

地址

指令又在哪儿呢?

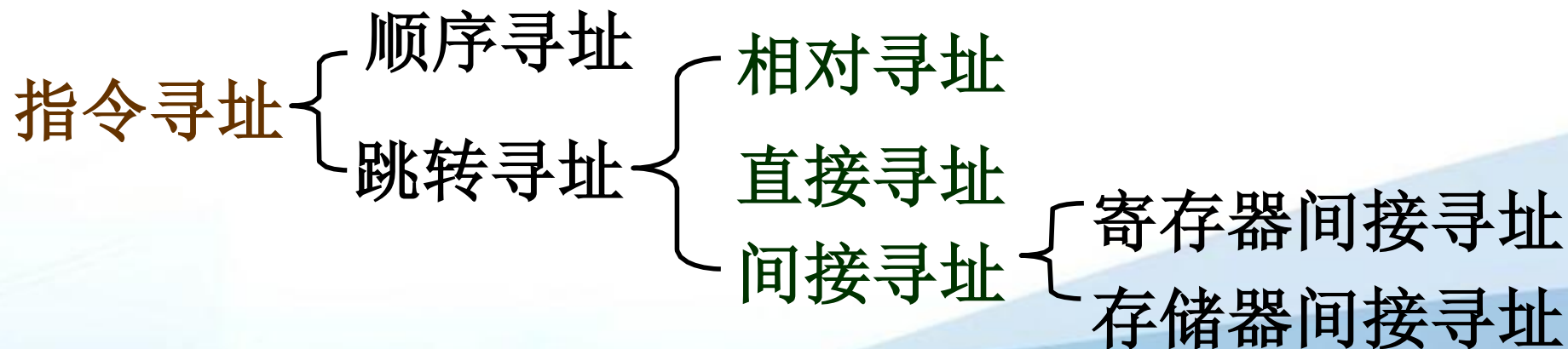
数据
数据
指令
指令

数据寻址的总结



指令寻址

- 顺序寻址：EIP自动增量指向下一条指令
 - 顺序执行接着的下一条指令
- 跳转寻址：控制流程跳转(转移)到指定指令位置
 - 实现程序分支、循环、调用等结构



指令的相对寻址

➤ 提供目标地址相对于当前指令指针EIP的位移量

目标地址(转移后的IP) = 当前EIP + 位移量

- ▶ 相对寻址都是段内转移
- ▶ 最常用、最灵活

操作码	位移量
-----	-----

+ 当前地址

目标地址

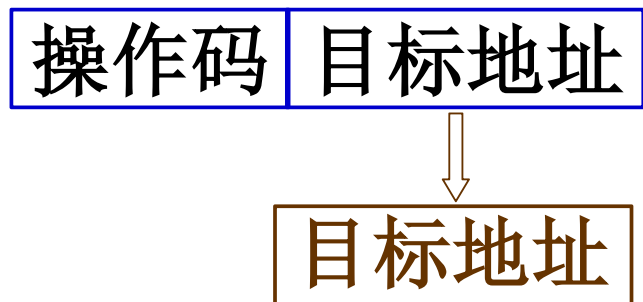
目标地址 = 目的地址 = 转移地址

指令的直接寻址

➤ 直接提供目标地址

目标地址 = 指令操作数

- ▶ 理论上可以段内或段间转移
- ▶ IA-32只支持段间的直接转移



指令的间接寻址

➤ 指示寄存器或存储单元

目标地址来自寄存器或存储单元、间接获得

- ▶ 寄存器间接寻址：用寄存器保存目标地址
- ▶ 存储器间接寻址：用存储单元保存目标地址

操作码 寄存器地址

寄存器

目标地址

操作码 存储器地址

存储单元

目标地址

程序流程的控制转移

➤ 程序代码在代码段

- ▶ **CS**: 指明代码段在主存中的段基地址
- ▶ **EIP**: 给出将要执行指令的偏移地址

➤ 指令顺序寻址，程序顺序执行

- ▶ 处理器自动增量**EIP**

➤ 指令跳转寻址，程序控制转移

- ▶ **EIP (CS)** 随之改变

控制转移类指令

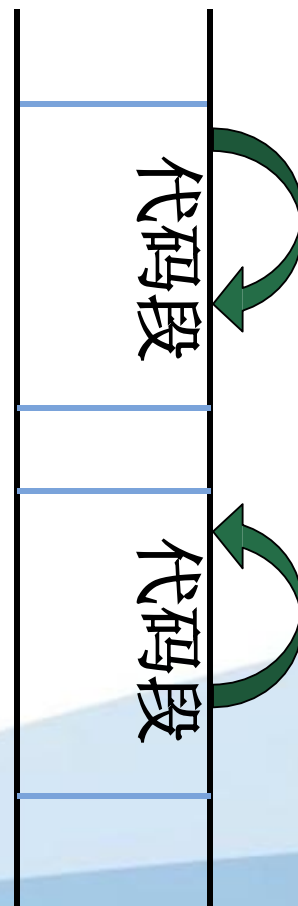
JMP

Jcc

LOOP

程序转移的范围：段内转移

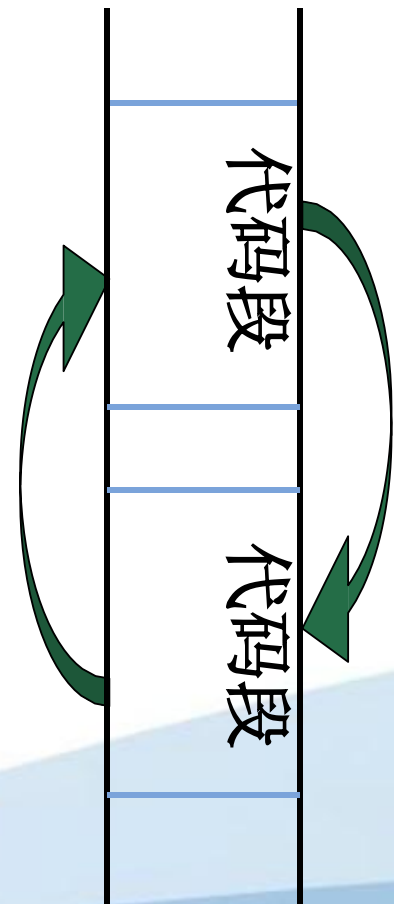
- 在当前代码段范围内的程序转移
 - 不需更改CS，只要改变EIP（偏移地址）
- 被称为“近转移”
 - 类型属性使用“NEAR”关键字
- 如果转移范围在127~-128字节之间，
（位移量使用1个字节）又称为“短转移”
 - 类型属性使用“SHORT”关键字



程序转移的范围：段间转移

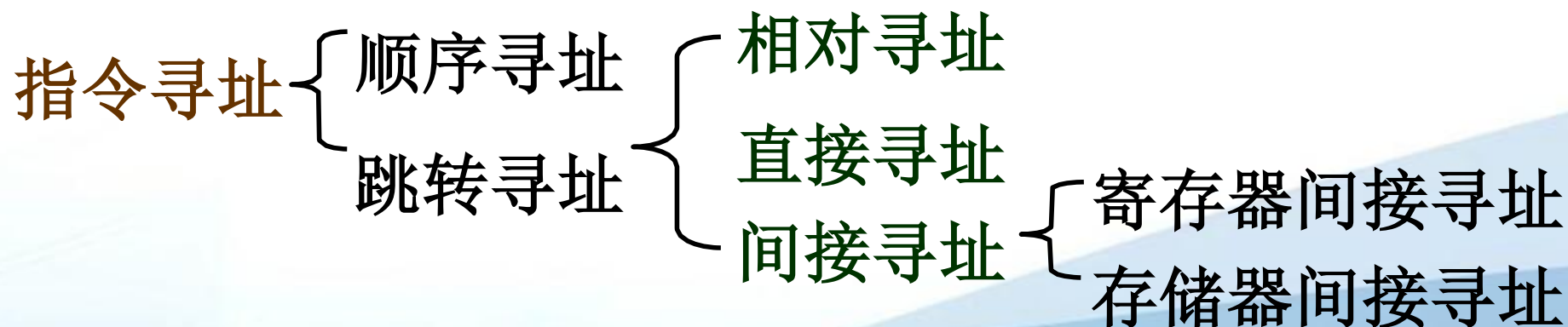
- 从当前代码段跳转到另一个代码段
- 需要更改CS（段地址）和EIP（偏移地址）
- 被称为“远转移”
 - ▶ 类型属性使用“FAR”关键字

标号、子程序名等具有的类型属性
NEAR SHORT FAR



指令寻址的总结

- IA-32处理器通过改变EIP（和CS）控制程序流程
 - 实现程序分支、循环、调用等结构
- 可以实现代码段内、或代码段间跳转
 - 具有短转移short、近转移near和远转移far属性





福州大学
FUZHOU UNIVERSITY

汇编语言程序设计

JMP指令

无条件转移指令 (JMP)

JMP label ;程序转向label标号指定的地址

;段内相对寻址, 段间直接寻址

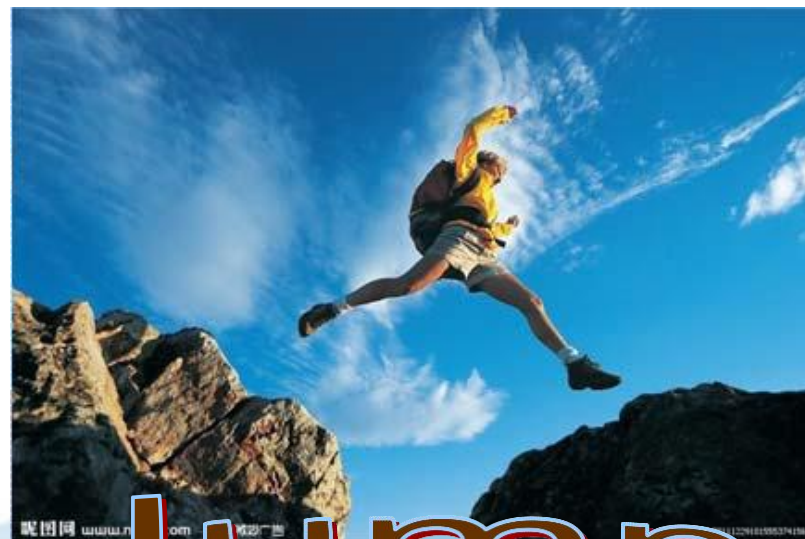
JMP reg32/reg16 ;程序转向寄存器指定的地址

;寄存器间接寻址

JMP mem48/mem32/mem16

;程序转向存储单元指定的地址

;存储器间接寻址



Jump

无条件转移程序—1

;代码段

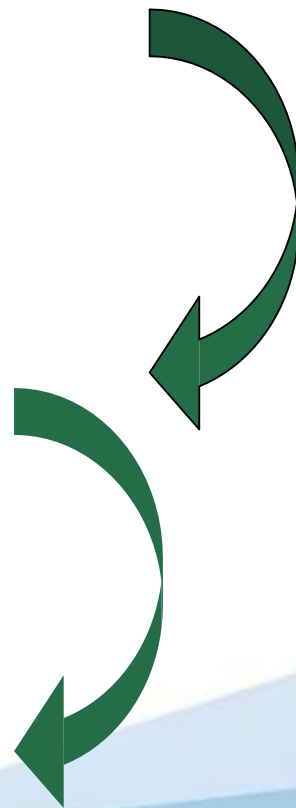
jmp labl1

nop

labl1: jmp near ptr labl2

nop

labl2: mov eax,offset labl3



无条件转移程序—1

;代码段

位移量, 1个字节

00000000 EB 01 jmp labl1

;相对寻址, 短转移

00000002 90 nop

00000003 E9 00000001

位移量, 4个字节

labl1: jmp near ptr labl2 ;相对寻址, 近转移

00000008 90 nop

00000009 B8 00000011 R

labl2: mov eax, offset labl3

无条件转移程序—2

labl2: mov **eax**,offset labl3

jmp **eax**

nop

labl3: mov **eax**,offset labl4



无条件转移程序—2

00000009 B8 00000011 R

labl2: mov eax,offset labl3

0000000E FF E0

jmp eax ;寄存器间接寻址

00000010 90

nop

00000011 B8 00000022 R

labl3: mov eax,offset labl4

无条件转移程序—3

labl3: mov eax,offset labl4

mov nvar,eax

jmp nvar

nop

labl4:

;数据段

nvar dword ?



无条件转移程序—3

labl3: mov eax,offset labl4

00000016 A3 00000000 R

mov nvar,eax

0000001B FF 25 00000000 R

jmp nvar

00000021 90

nop

labl4:

;数据段

nvar dword ?

数据的存储器直接寻址

;存储器间接寻址

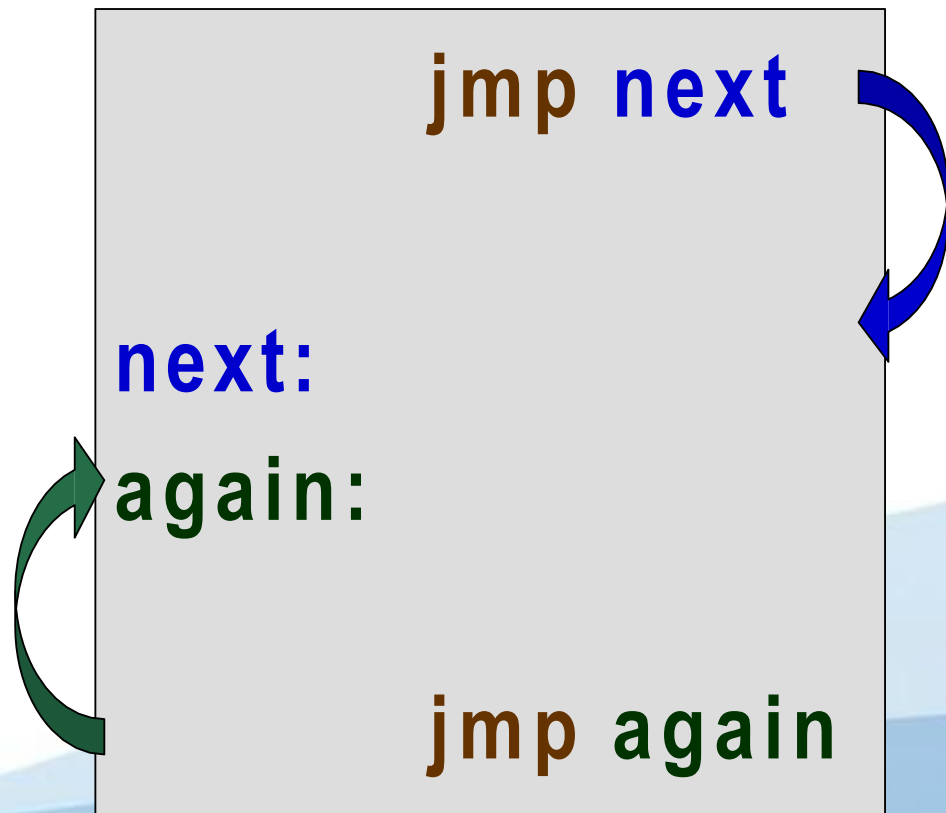
mov ebx,offset nvar

jmp near ptr [ebx]

;数据的寄存器间接寻址

本讲总结

- **JMP** 指令实现无条件的程序流程转移
- 对应C语言的goto语句
- 高级语言慎用goto语句
- 处理器必不可少**JMP**指令



汇编语言程序设计

4.2.2 条件转移指令

汇编语言程序设计

Jcc指令

条件转移

- 根据指定的条件确定程序是否发生转移

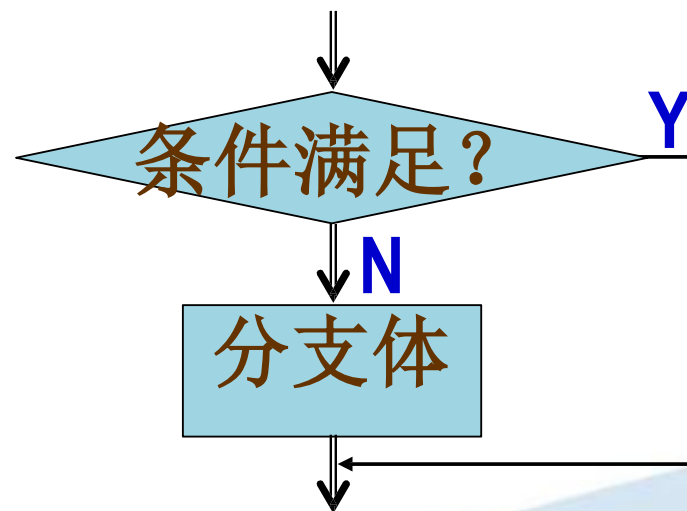
Jcc label

; 条件满足，发生转移；

; 否则，顺序执行下条指令

- label 表示目标地址
采用段内相对寻址

Jcc: jump with condition



判断的标志条件cc

➤ 共16条指令，分成两类

▶ 单个标志状态作为条件 5个状态标志ZF、CF、SF、OF和PF的10种状态

▶ 两数大小关系作为条件

• 比较无符号整数大小 4种情况：低于、不低于、低于等于、高于

• 比较有符号整数大小 4种情况：小于、不小于、小于等于、大于

利用零位标志ZF的条件转移指令

➤判断条件：运算结果为0、两数相等（标志ZF=1）

JZ label ;Jump if Zero

JE label ;Jump if Equal

➤判断条件：结果不为0、不相等（标志ZF=0）

JNZ label ;Jump if Not Zero

JNE label ;Jump if Not Equal

多个助记符方便记忆

利用进位标志CF的条件转移指令

➤判断条件：运算结果有进位(借位) (标志CF=1)

JC label ;Jump if Carry

➤判断条件：结果没有进位(借位) (标志CF=0)

JNC label ;Jump if Not Carry

JC还等于JB和JNAE指令

JNC还等于JNB和JAE指令

利用溢出标志OF的条件转移指令

➤判断条件：运算结果有溢出（标志OF=1）

JO label ;Jump if Overflow

➤判断条件：结果没有溢出（标志OF=0）

JNO label ;Jump if Not Overflow

溢出标志OF针对有符号整数加减运算
进位标志CF针对无符号整数加减运算

利用符号标志SF的条件转移指令

➤判断条件：运算结果是负、最高位为1（标志SF=1）

JS label ;Jump if Sign

➤判断条件：结果是正、最高位为0（标志SF=0）

JNS label ;Jump if Not Sign

有符号整数采用补码，最高位是符号位
符号位为1，表示负数；符号位为0，表示正数

利用奇偶标志PF的条件转移指令

➤判断条件：低8位结果中1的个数为偶或0（标志PF=1）

JP label ;Jump if Parity

JPE label ;Jump if Parity Even

➤判断条件：低8位结果中1的个数为奇（标志PF=0）

JNP label ;Jump if Not Parity

JPO label ;Jump if Parity Odd

多个助记符方便记忆

两个无符号整数大小关系的条件转移指令-1

➤判断条件：低于、不高于等于（标志CF=1）

JB ;Jump if Below

JNAE ;Jump if Not Above or Equal

➤判断条件：不低于、高于等于（标志CF=0）

JNB ;Jump if Not Below

JAE ;Jump if Above or Equal

JC

JNC

无符号数大小用高(Above)、低(Below)助记符

两个无符号整数大小关系的条件转移指令-2

➤判断条件：低于等于、不高于（标志CF=1或ZF=1）

JBE ;Jump if Below or Equal

JNA ;Jump if Not Above

➤判断条件：不低于等于、高于（标志CF=0且ZF=0）

JNBE ;Jump if Not Below or Equal

JA ;Jump if Above

无符号数大小用高(Above)、低(Below)助记符

两个有符号整数大小关系的条件转移指令-1

➤判断条件：小于、不大于等于（标志SF≠0F）

JL ;Jump if Less

JNGE ;Jump if Not Greater or Equal

➤判断条件：不小于、大于等于（标志SF=0F）

JNL ;Jump if Not Less

JGE ;Jump if Greater or Equal

有符号数大小用大(Greater)、小(Less)助记符

两个有符号整数大小关系的条件转移指令-2

➤判断条件：小于等于、不大于(标志 $SF \neq 0$ 或 $ZF=1$)

JLE ;Jump if Less or Equal

JNG ;Jump if Not Greater

➤判断条件：不小于等于、大于(标志 $SF=0$ 且 $ZF=0$)

JNLE ;Jump if Not Less or Equal

JG ;Jump if Greater

有符号数大小用大(Greater)、小(Less)助记符

产生条件的指令

➤常用指令1：比较指令CMP

- 进行减法运算
- 用于判断两个数据大小、是否相等

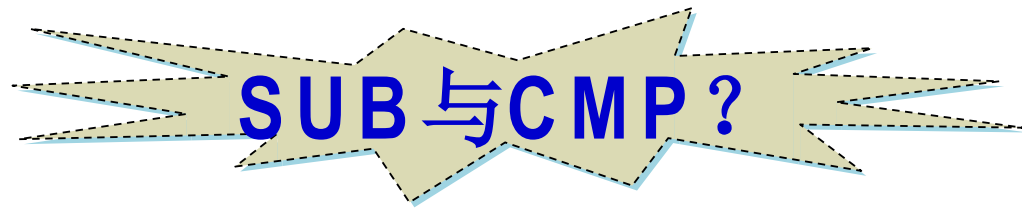
➤常用指令2：测试指令TEST

- 进行逻辑与运算
- 用于判断某位为0或为1等

➤其他指令：能够影响状态标志的指令 加减运算指令、逻辑运算指令、移位指令等

比较指令CMP (compare)

- 将目的操作数减去源操作数
 - 差值不回送目的操作数
 - 按照减法结果影响状态标志



CMP reg,imm/reg/mem ;reg—imm/reg/mem

CMP mem,imm/reg ;mem—imm/reg

- 根据标志状态获知两个操作数的大小关系
- 给条件转移等指令使用其形成的状态标志

测试指令TEST

- 按位进行逻辑与运算，不返回逻辑与结果

TEST reg,imm/reg/mem ;reg $\wedge$ imm/reg/mem

TEST mem,imm/reg ;mem $\wedge$ imm/reg

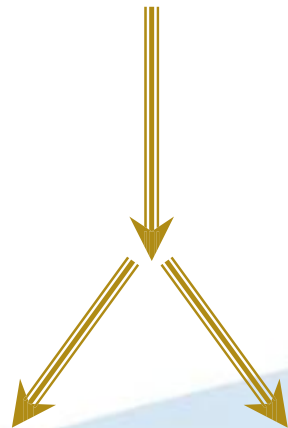
- **TEST**指令像**AND**指令一样来设置状态标志
- 常用于检测一些条件是否满足，一般后跟条件转移指令，目的是利用测试条件转向不同的分支

AND与TEST?

TEST与CMP?

本讲总结

- 条件转移指令
 - ▶ 在满足条件的情况才实现转移
 - ▶ 条件不满足，则顺序执行
- 利用Jcc指令实现分支、循环程序结构
- 其作用对应C语言的if语句



Jcc跳转指令汇总(16条)

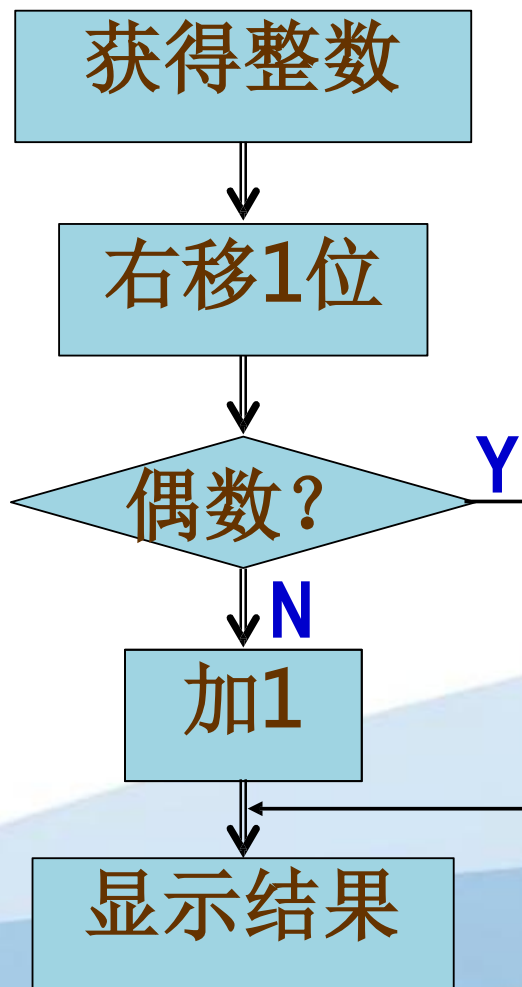
条件分类	标志位	标志位	指令	助记单词	指令全称
单标志位	ZF	ZF=1 ZF=0	JZ JNZ	Zero	Jump if Zero Jump if Not Zero
	CF	CF=1 CF=0	JC JNC	Carry Flag	Jump if Carry Jump if Not Carry
	SF	SF=1 SF=0	JS JNS	Sign	Jump if Sign Jump if Not Sign
	OF	OF=0 OF=1	JO JNO	Overflow	Jump if Overflow Jump if Not Overflow
	PF	PF=1 PF=0	JP JPE JNP JPO	Parity	Jump if Parity Jump if Parity Even Jump if Not Parity Jump if Parity Odd
两数大小 关系	无符号数: 低于、不高于等于 不低于、高于等于 低于等于、不高于 不低于等于、高于	CF=1 CF=0 CF=1或ZF=1 CF=0且ZF=0	JB/JNAE JNB/JAE JBE/JNA JNBE/JA	Above Below Equal	Jump if Below / jump if Not Above or Equal Jump if Not Below / Jump if Above or Equal Jump if Below or Equal / Jump if Not Above Jump if Not Below or Equal / Jump if Above
	有符号数: 小于、不大于等于 不小于、大于等于 小于等于、不大于 不小于等于、大于	SF≠OF SF=OF SF≠O或ZF=1 SF=O且ZF=0	JL/JNGE JNL/JGE JLE/JNG JNLE/JG	Greater Less	Jump if Less / Jump if Not Greater or Equal Jump if Not Less / Jump if Greater or Equal Jump if Less or Equal / Jump if Not Greater Jump if Not Less or Equal / Jump if Greater

汇编语言程序设计

个数折半程序

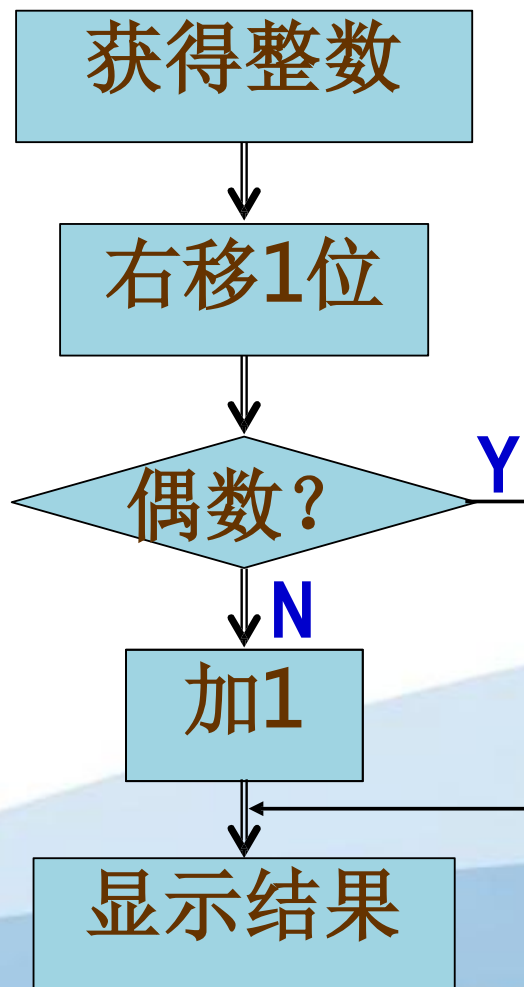
个数折半

- 对一个（无符号）整数折半
 - 就是对数据除以2
 - 数据右移1位实现除以2更方便
- 如果是偶数，完成
- 如果是奇数，需要加1
- 显示折半后的结果



偶数判断

- (SHR指令) 右移1位对整数折半
- 如果是偶数、最低位是0
 - 即，移入CF标志的位为0，完成
- 如果是奇数，需要加1
- 显示折半后的结果



条件转移指令 (Jcc)

- 根据指定的条件确定程序是否发生转移

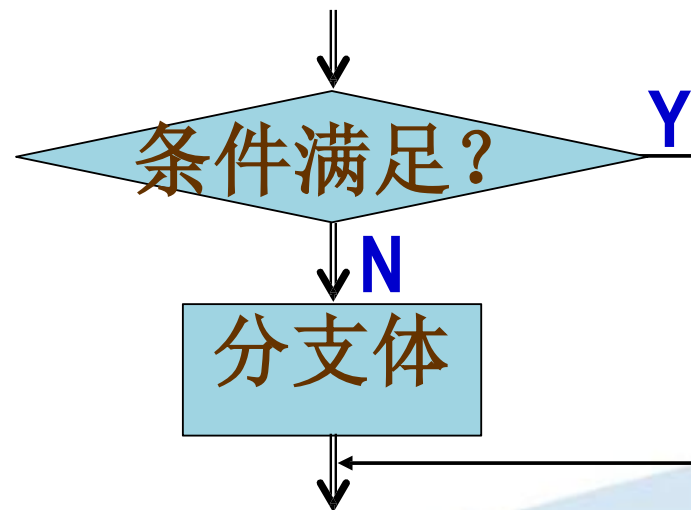
Jcc label

; 条件满足, 发生转移;

; 否则, 顺序执行下条指令

- label 表示目标地址

采用段内相对寻址



Jcc: jump with condition

利用进位标志CF的条件转移指令

➤判断条件：运算结果有进位(借位) (标志CF=1)

JC label ;Jump if Carry

➤判断条件：结果没有进位(借位) (标志CF=0)

JNC label ;Jump if Not Carry

判断CF=0，选用JNC指令

个数折半程序

mov eax,885

shr eax,1 ;右移1位

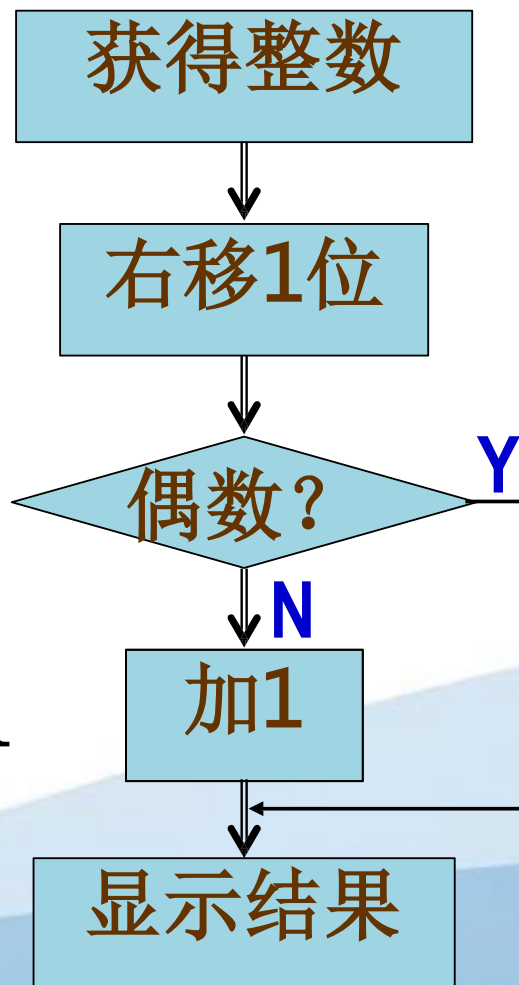
jnc goeven

;CF=0条件成立, 转移

add eax,1

;条件不成立 (CF=1), 加1

goeven: call dispuid ;显示结果



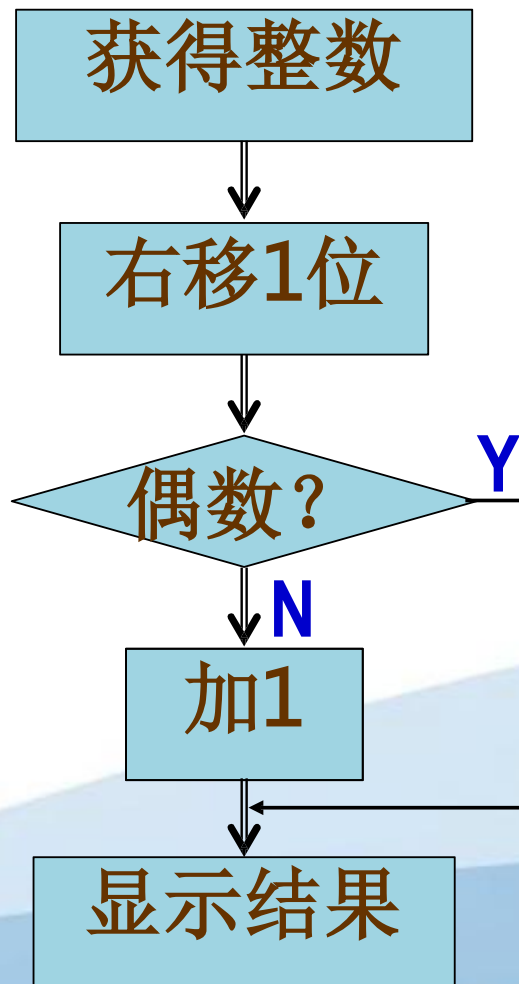
误用高级语言思维

➤ 习惯使用高级语言if语句

`if (CF标志=1) 加1;`

➤ 于是，选用JC指令

判断CF=1，选用JC指令



个数折半程序（误用JC）

```
mov eax,886
```

```
shr eax,1    ;右移1位
```

```
jc goodd
```

```
;CF=1条件成立，转移
```

```
jmp goeven    ;CF=0，转移到显示！
```

```
goodd:  add eax,1    ;加1
```

```
goeven:  call dispuid    ; 显示结果
```



个数折半程序（误用JC）

```
mov eax,886
```

```
shr eax,1 ;右移1位
```

```
jnc goeven
```

```
jc goodd
```

```
;CF=1条件成立，转移
```

```
jmp goeven
```

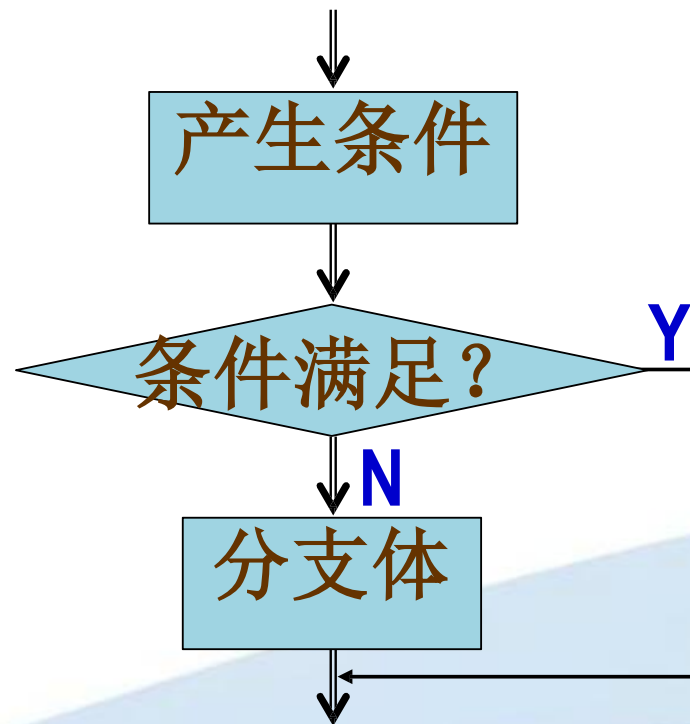
```
;CF=0，转移到显示！
```

```
goodd: add eax,1 ;加1
```

```
goeven: call dispuid ; 显示结果
```


本讲总结

- 分析应用问题
- 考虑产生条件的指令
- 选择最适合的条件转移指令
- 形成分支程序结构





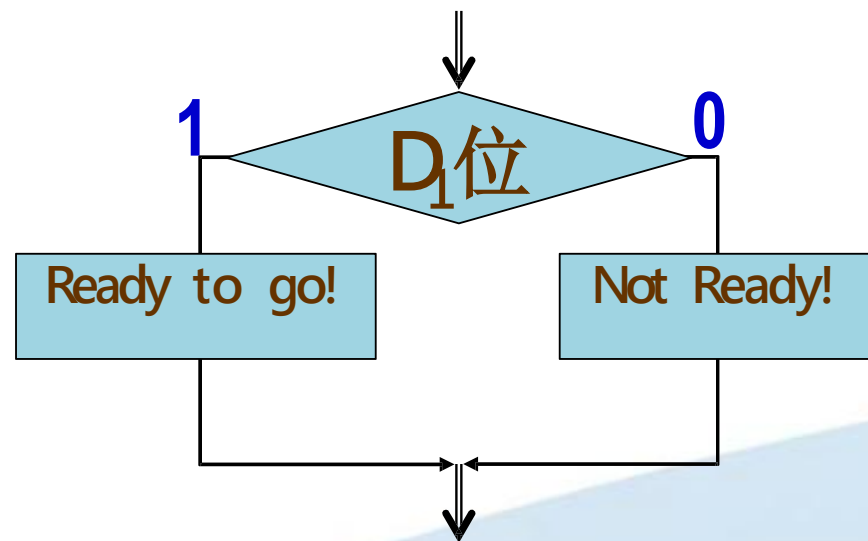
福州大学
FUZHOU UNIVERSITY

汇编语言程序设计

位测试程序

位测试

- 底层程序设计，常需测试某个位（bit）为0或为1
 - ▶ 为0，表示一种状态
 - ▶ 为1，表示相反状态
- 例如，测试数据D₁位为0或为1
 - ▶ 为0，显示：Not Ready !
 - ▶ 为1，显示：Ready to go !



```
no_msg byte 'Not Ready!',0  
yes_msg byte 'Ready to go!',0
```

如何进行位测试

- 使用位操作类指令：逻辑指令、移位指令等
- 典型应用：使用测试指令（**TEST**）
 - ▶ 将要测试位除外的其他位“逻辑与”为0，这样
 - “逻辑与”结果为0，表示测试位为0
 - “逻辑与”结果为1，表示测试位为1
 - ▶ 使用零标志条件转移指令（**JZ JNZ**）进行分支



位测试程序（选用JZ指令）

;代码段

```
mov eax,56h      ;假设一个数据
test eax,02h     ;测试D1位(D1=1, 其他位为0)
jz nom           ;D1=0条件成立, 转移
mov eax,offset yes_msg ;D1=1, 准备好
jmp done         ;跳转过另一个分支体!
nom:             ;没有准备好
mov eax,offset no_msg
done:            call dispmsg ;显示信息
```



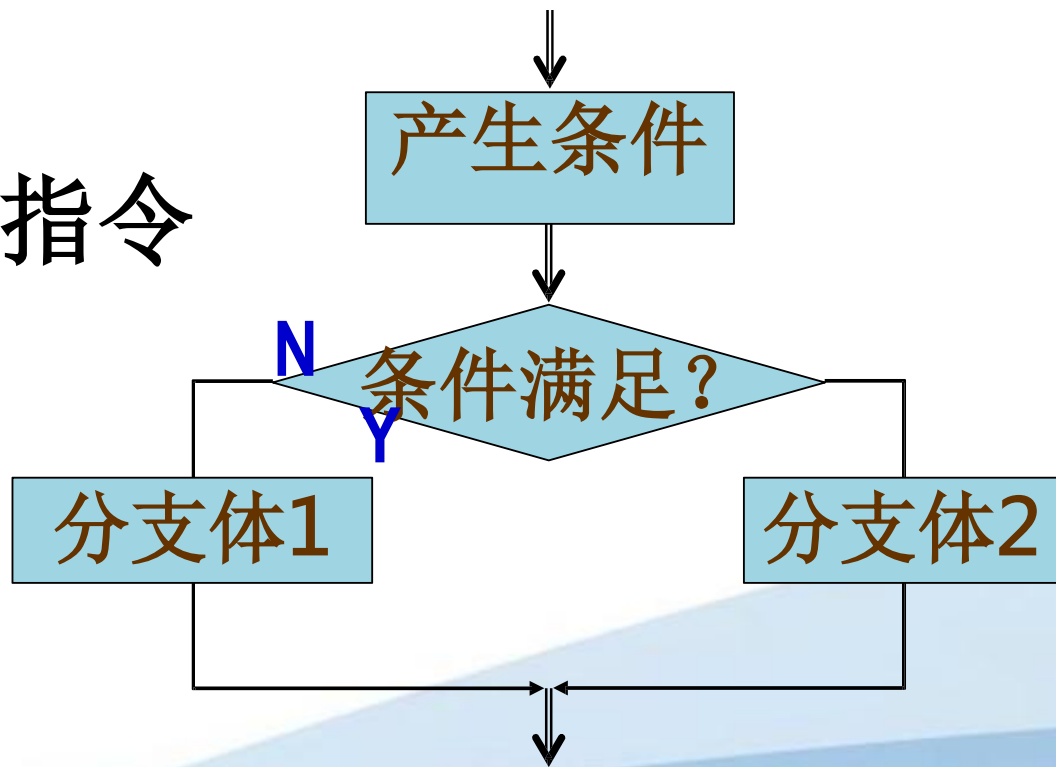
位测试程序（选用JNZ指令）

;代码段

```
mov eax,56h      ;假设一个数据
test eax,02h     ;测试D1位 (D1=1, 其他位为0)
jnz yesm         ;D1=1条件成立, 转移
mov eax,offset no_msg ;D1=0, 没有准备好
jmp done         ;跳转过另一个分支体!
yesm: mov eax,offset yes_msg ;准备好
done:  call dispmsg    ;显示信息
```

本讲总结

- 分析应用问题
- 考虑产生条件的指令
- 选择最适合的条件转移指令
- 形成分支程序结构



汇编语言程序设计

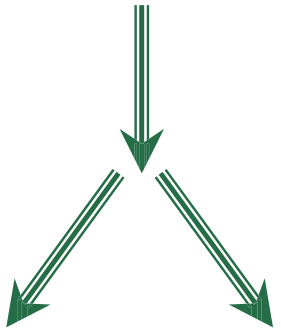
单分支结构

分支程序结构

➤ 有条件产生和条件判断两个部分组成

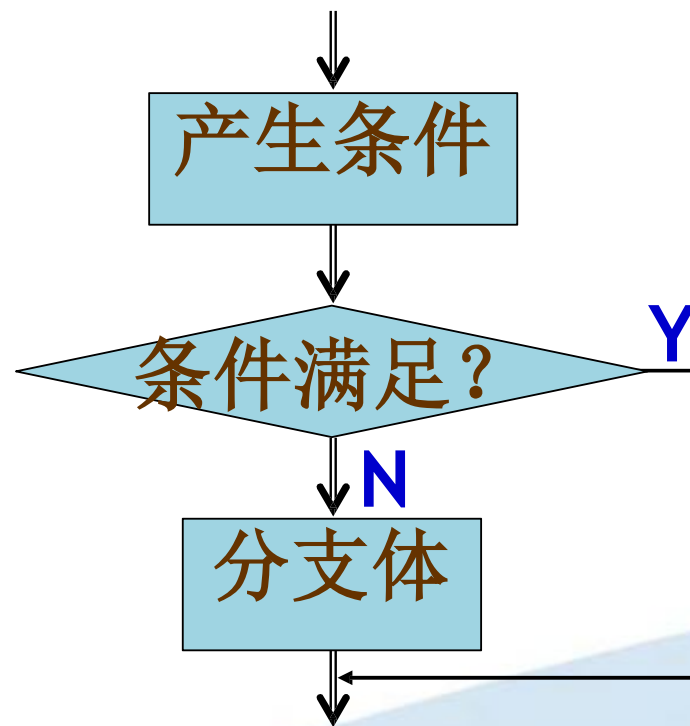
▶ 首先，利用比较CMP、测试TEST、或者加减运算、逻辑运算等影响状态标志的指令——形成条件；

▶ 然后，利用条件转移指令判断由标志表达的条件——并根据标志状态控制程序转移到不同的程序段。

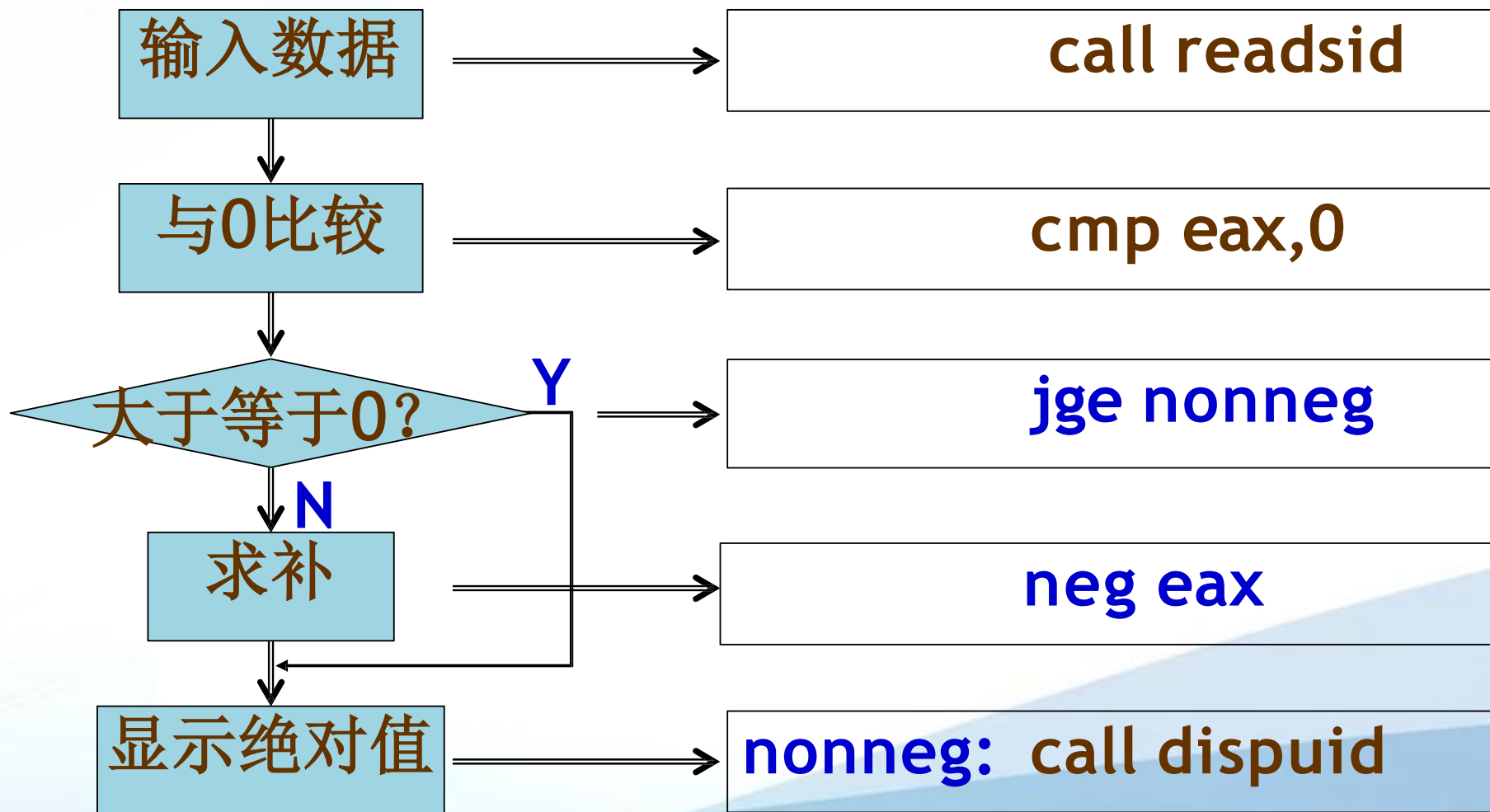


单分支程序结构

- 只有一个分支的程序
 - 类似if-then语句
- 当条件满足（成立）
 - 发生转移，跳过分支体
- 当条件不满足
 - 顺序向下执行分支体



求绝对值





求绝对值程序

```
call readsid
```

```
cmp eax,0
```

```
jge nonneg
```

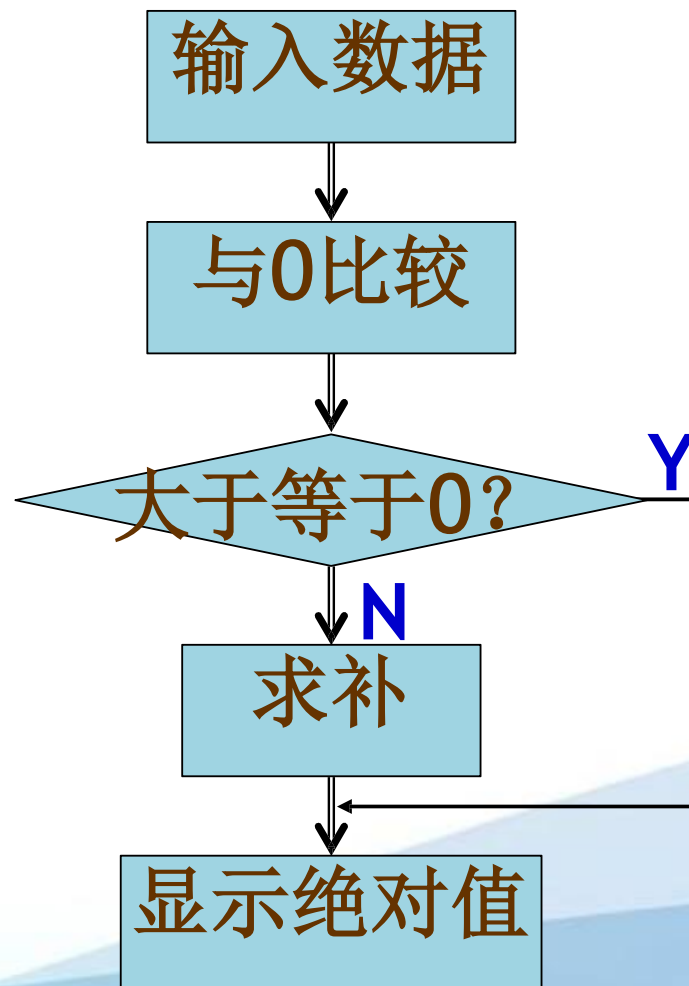
```
neg eax
```

```
nonneg: call dispuid
```

-2468

2468

运行示例





求绝对值程序

```
call readsid      ;输入有符号数，从EAX返回值  
cmp eax,0         ;比较EAX与0  
jge nonneg        ;条件满足 ( $EAX \geq 0$ )，转移  
neg eax           ;条件不满足 ( $EAX < 0$ )，负数求补得正值  
nonneg: call dispuid ;分支结束，显示无符号结果
```

顺序执行的部分是分支体

求绝对值（误用Jcc指令）

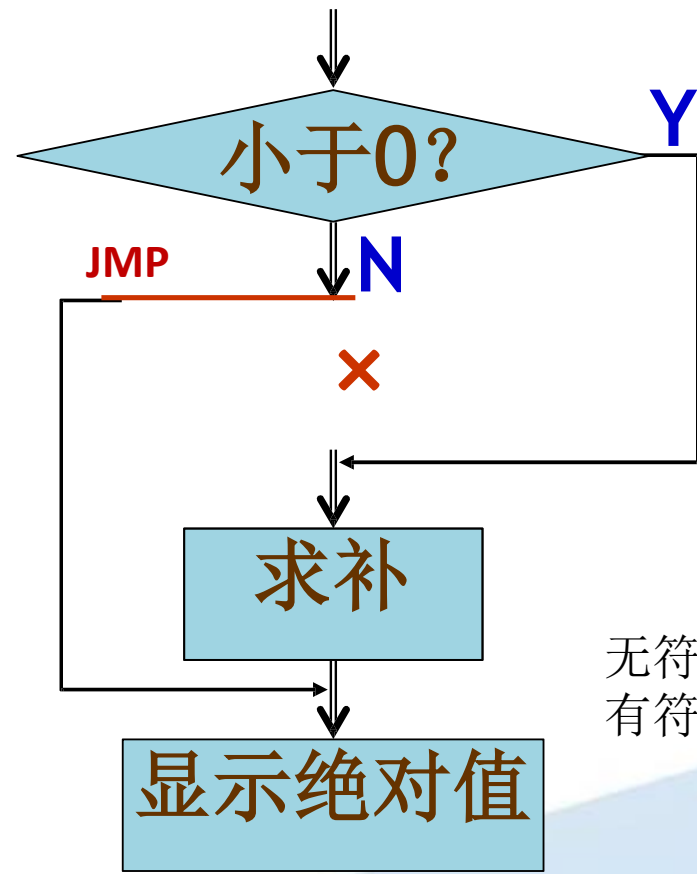
```
call readsid  
cmp eax,0
```

```
jl yesneg  
jmp nonneg
```

```
yesneg: neg eax
```

```
nonneg: call dispuid
```

```
jge nonneg
```



JMP 指令条件满足，
跳转了；不满足顺
序执行。

与ifthen，逻辑
相反

无符号数：Above Below
有符号数：Greater Less

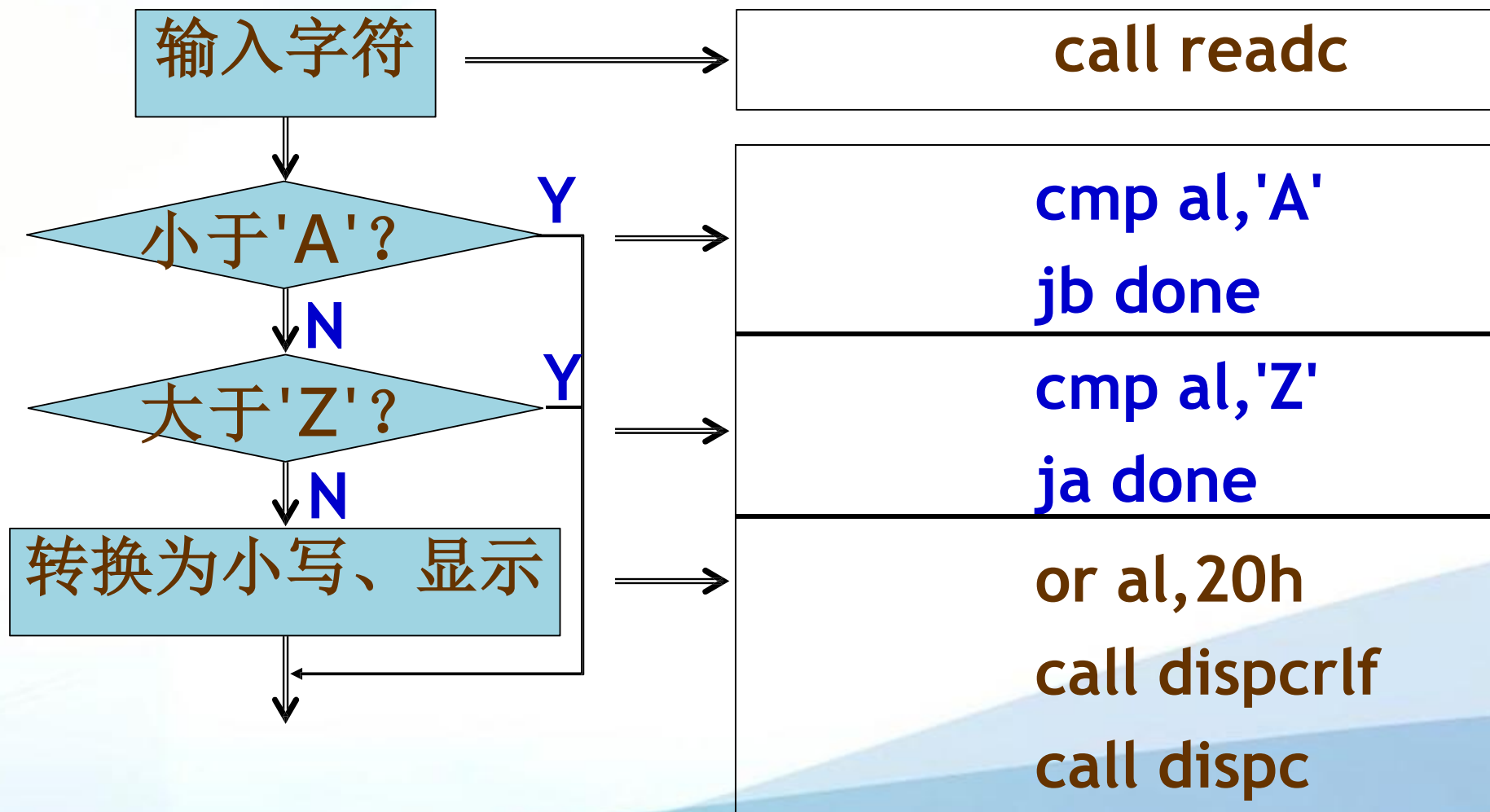
编写单分支程序结构的注意事项

- 只有一个分支的程序
 - ▶ 类似高级语言的if-then语句结构
- 注意采用正确的条件转移指令
 - ▶ 当条件满足（成立），发生转移，跳过分支体
 - ▶ 条件不满足，顺序向下执行分支体
- 条件转移指令与高级语言的if语句正好相反
 - ▶ if语句是条件成立，执行分支体

汇编语言程序设计

字母判断程序

大写字母判断

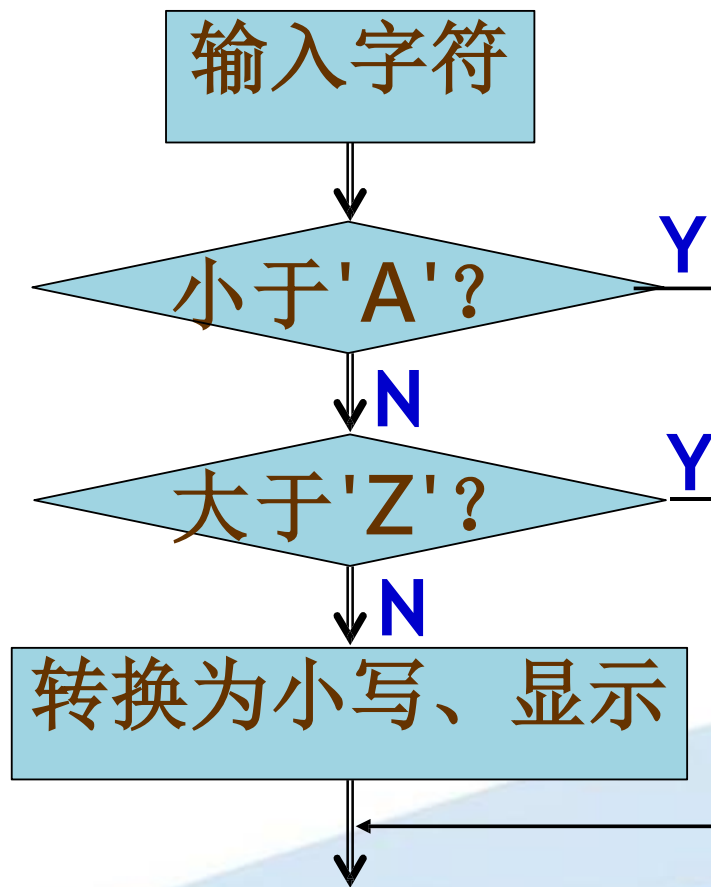




大写字母判断程序

```
call readc  
cmp al,'A'  
jb done  
cmp al,'Z'  
ja done  
or al,20h  
call dispCrLf  
call dispC
```

done:



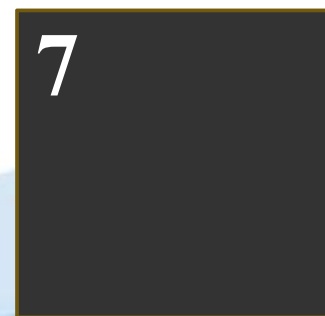
大写字母判断程序

call readc	;输入一个字符，从AL返回值
cmp al,'A'	;与大写字母A比较
jb done	;比大写字母A小，不是大写，转移
cmp al,'Z'	;与大写字母Z比较
ja done	;比大写字母Z大，不是大写，转移
or al,20h	;转换为小写
call dispCrLf	
call dispC	



Box showing uppercase 'H' and lowercase 'h'.

运行示例1

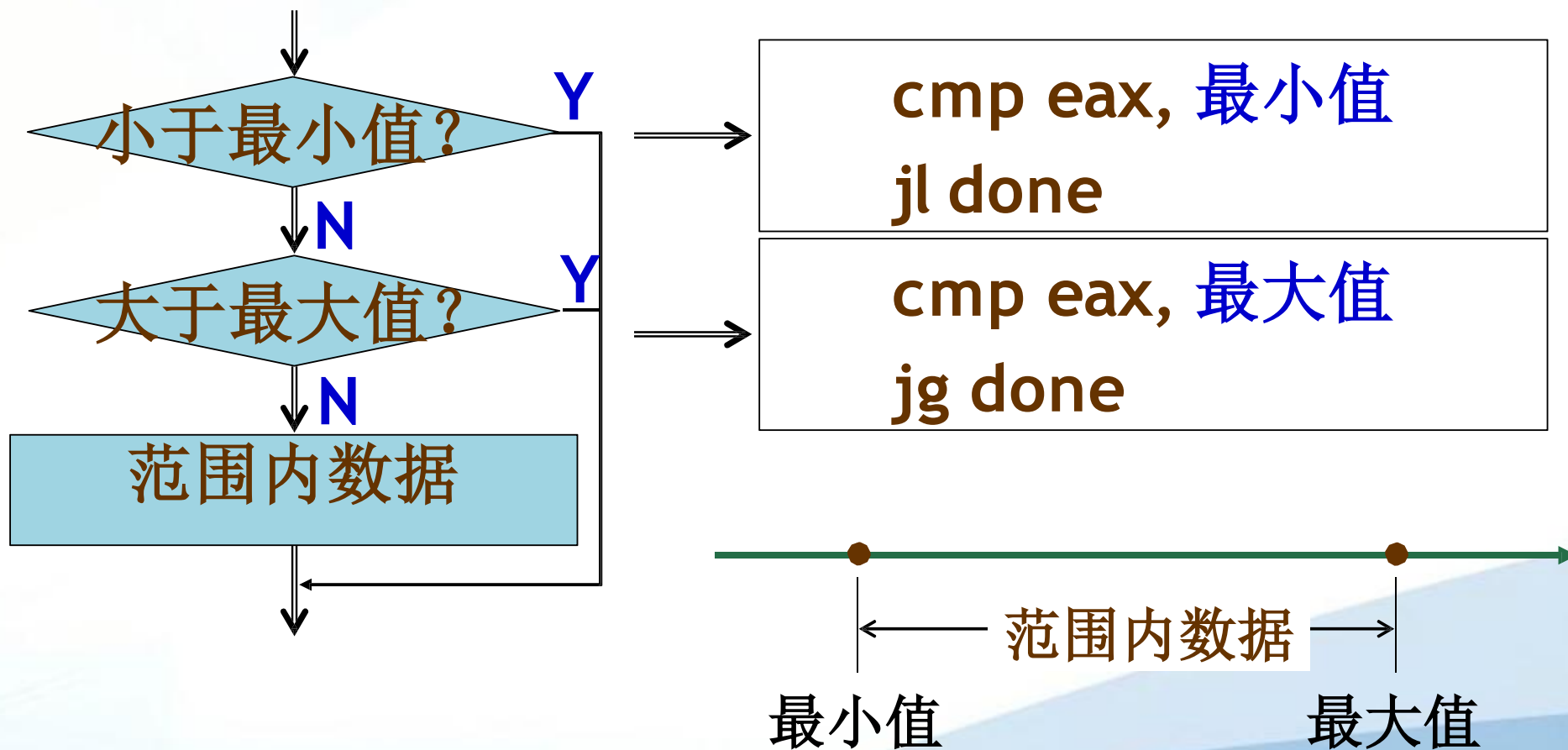


Box showing the number 7.

运行示例2

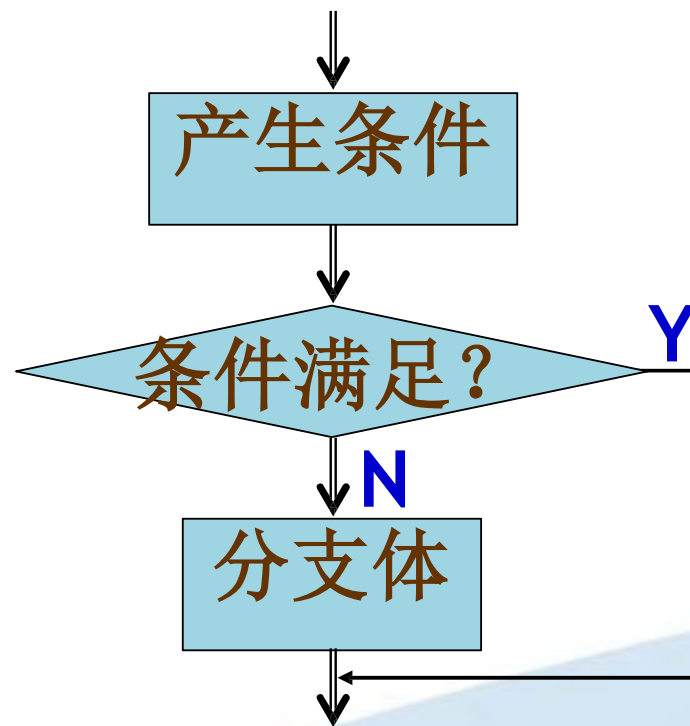
done:

数据范围的通用判断方法



单分支程序结构

- 当条件满足（成立）
 - 发生转移，跳过分支体
- 当条件不满足
 - 顺序向下执行分支体



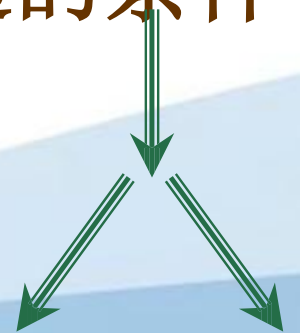
选择正确的条件转移指令

汇编语言程序设计

4.2.4 双分支结构

分支程序结构

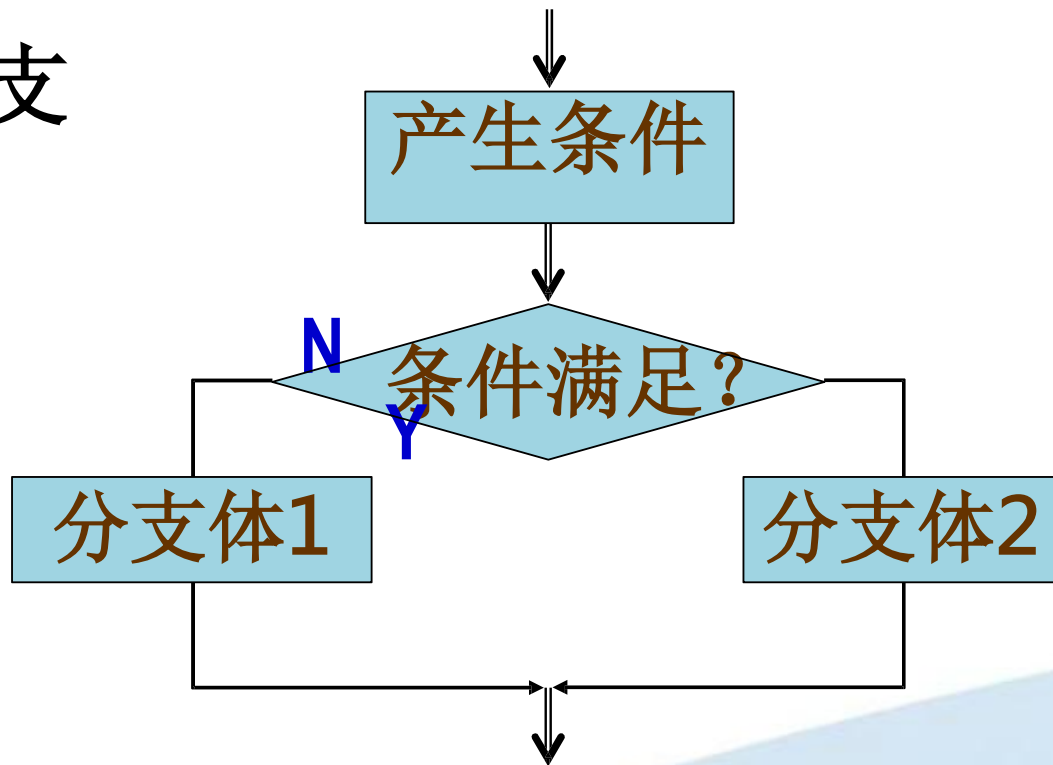
- 有条件产生和条件判断两个部分组成
 - ▶ 首先，利用比较CMP、测试TEST、或者 加减运算、逻辑运算等影响状态标志的指令 形成条件
 - ▶ 然后，利用条件转移指令判断由标志表达的条件并根据标志状态控制程序转移到不同的程序段



双分支程序结构

➤ 双分支程序结构有两个分支

- ▶ 条件为真，转移
执行分支体2
- ▶ 条件为假，顺序
执行分支体1



分支体1最后一定有JMP指令跳过分支体

条件转移指令 (Jcc)

- 根据指定的条件确定程序是否发生转移

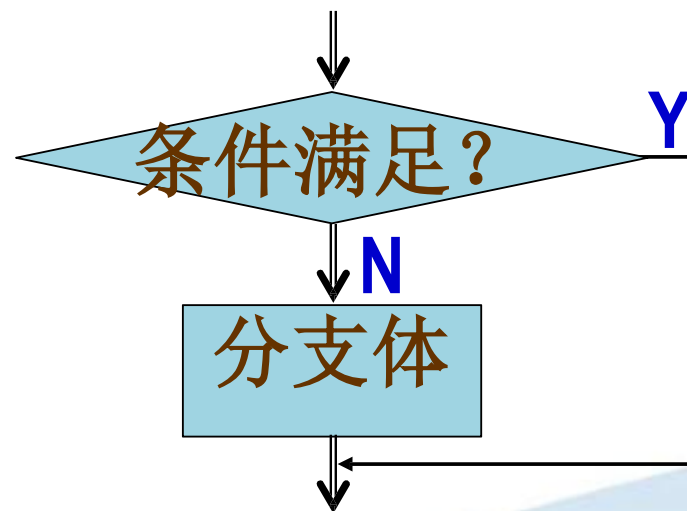
Jcc label

; 条件满足, 发生转移;

; 否则, 顺序执行下条指令

- label 表示目标地址

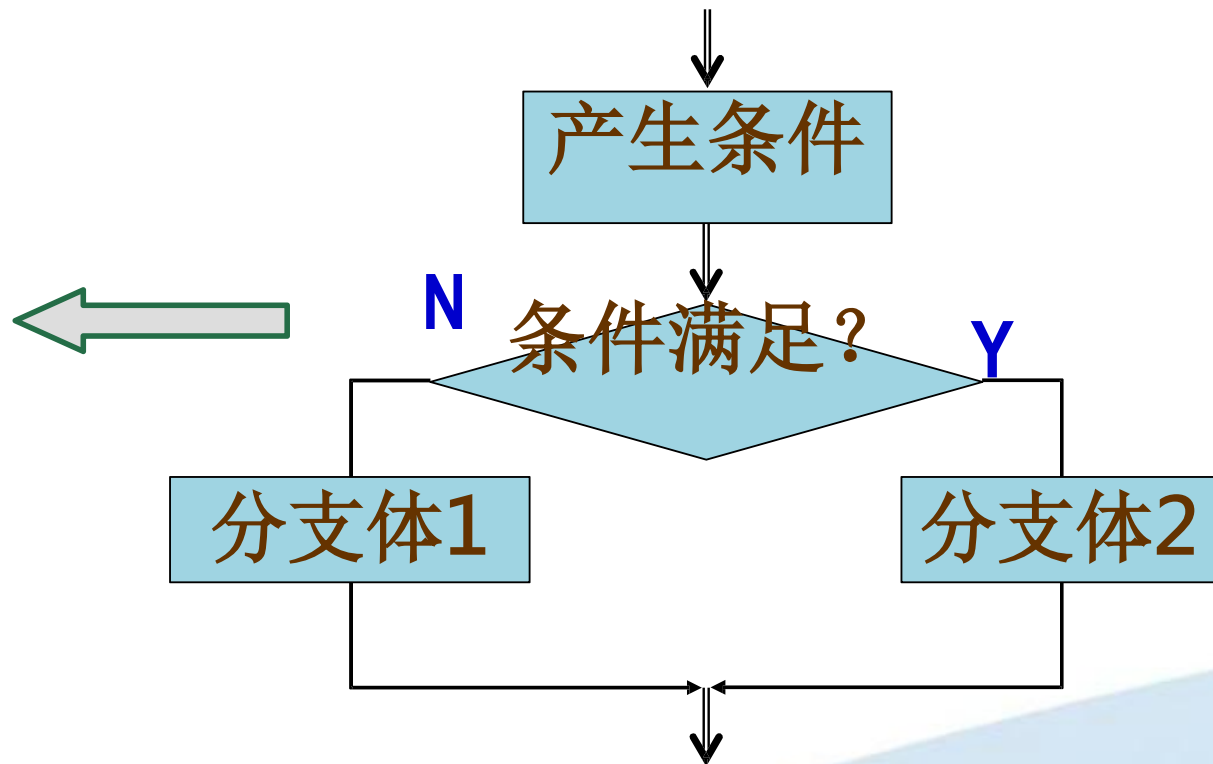
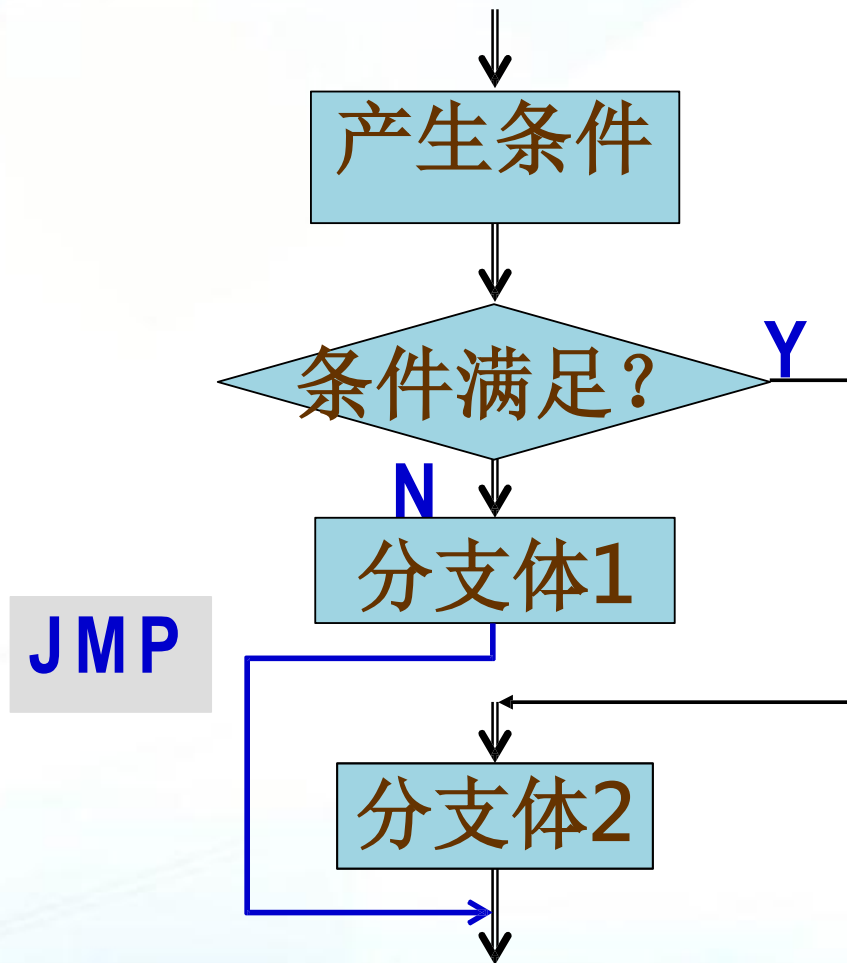
采用段内相对寻址



Jcc: jump with condition



顺序执行的分支体最后一定有JMP指令



显示数据最高位程序

```
mov ebx,dvar ;数据来自变量DVAR
shl ebx,1    ;EBX最高位移入CF标志
jc one       ;CF=1, 即最高位为1, 转移
mov al,'0'   ;CF=0, 即最高位为0:
jmp two      AL←'0'
one: mov al,'1'
two:  call dispc ;一定要跳过另一个分支
      ;AL←'1'
      ;显示
```

```
dvar dword 0bd980613h
```

显示数据最高位程序（选用JC）

```
mov ebx, dvar
```

```
shl ebx, 1
```

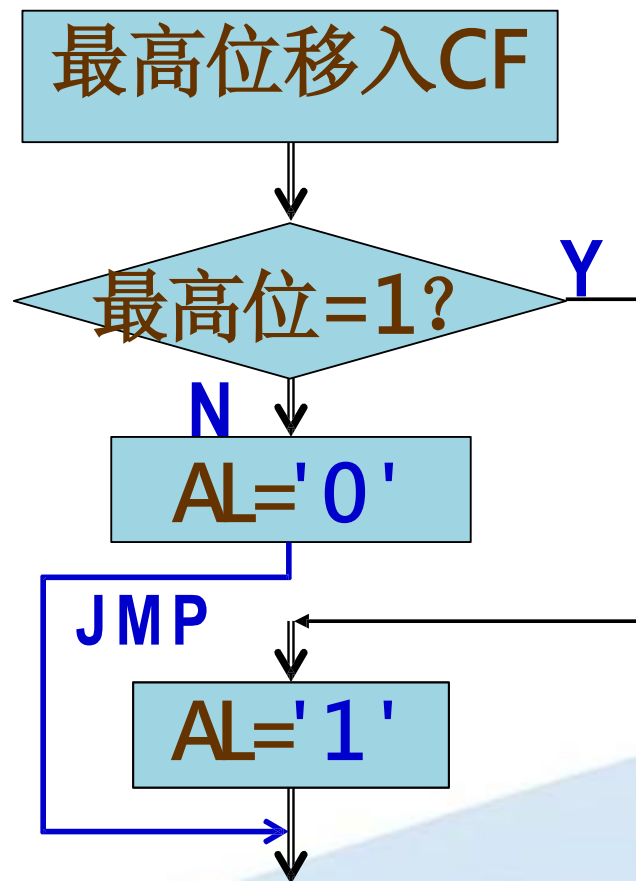
```
jc one
```

```
mov al, '0'
```

```
jmp two
```

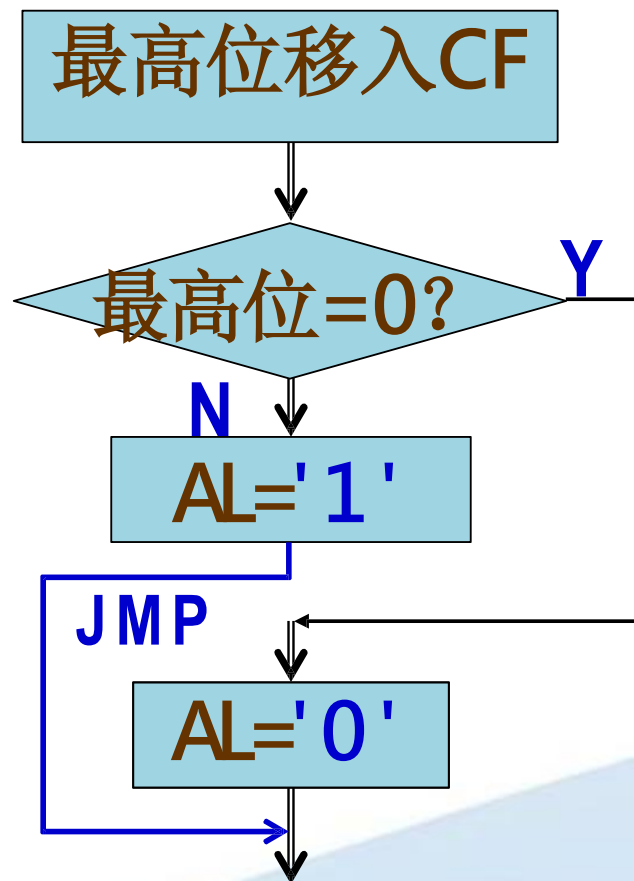
```
one: mov al, '1'
```

```
two: call disp
```



显示数据最高位程序（选用JNC）

```
mov ebx, dvar  
shl ebx, 1  
jnc one  
mov al, '1'  
jmp two  
one: mov al, '0'  
two: call dispc
```



编写双分支程序结构的注意事项

- 双分支程序结构有两个分支
 - 相当于高级语言的if-then-else语句
- 条件为真执行一个分支（分支2）
 - 条件为假，执行另一个分支（分支1）
- 顺序执行的分支体最后有JMP指令跳过另一个分支体（分支2）
 - JMP指令必不可少
 - 实现结束前一个分支回到共同的出口作用

汇编语言程序设计

有符号整数运算溢出程序



➤ 无符号整数，表达0和正整数

▶ 采用二进制直接编码

➤ 有符号整数，表达负整数、0和正整数

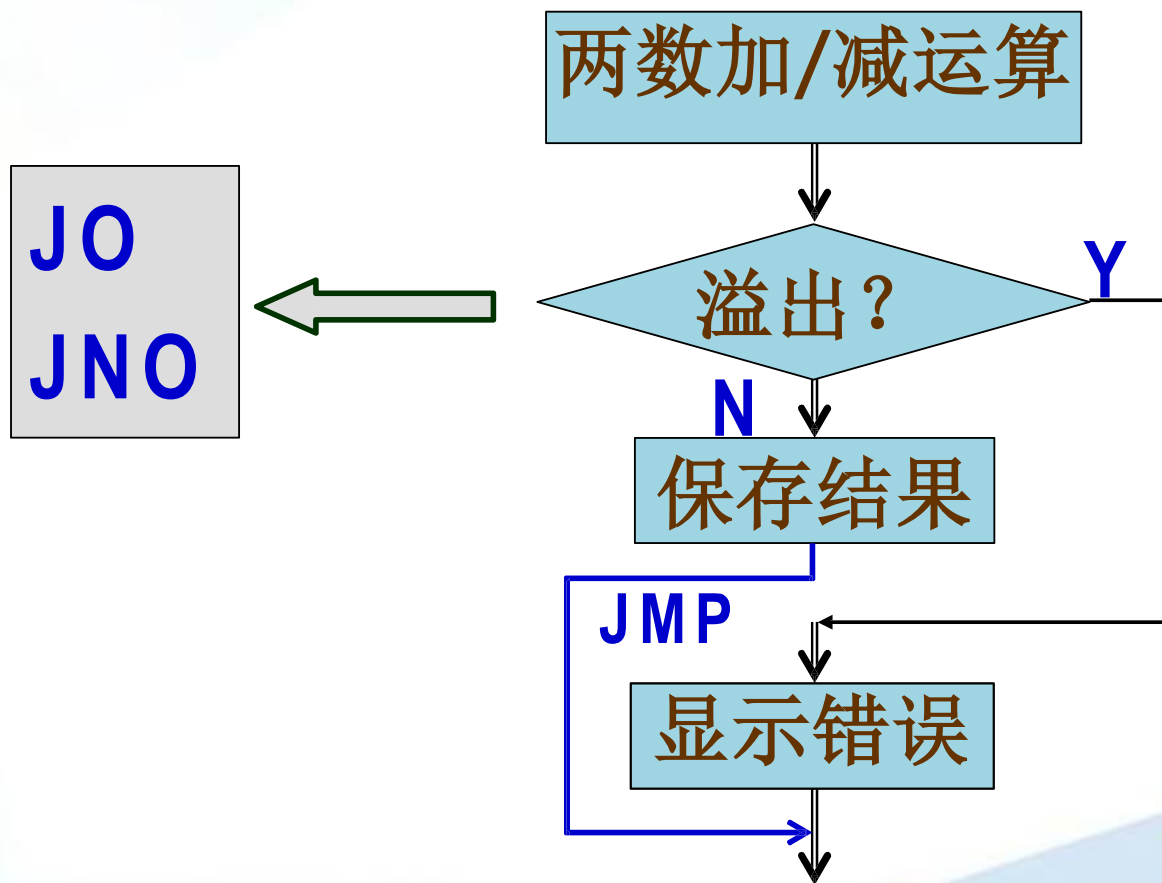
▶ 采用补码编码

配合CF标志

配合OF标志

指令	无符号整数	有符号整数
加减法	ADD/SUB	ADD/SUB
乘除法	MUL/DIV	IMUL/IDIV
条件转移	JA/JB	JG/JL

有符号数加减运算的溢出判断



两个有符号整数变量

;数据段

dvar1 dword 1234567890 ;假设两个数据

dvar2 dword -999999999

dvar3 dword ? ;保存结果的变量

okmsg byte 'Correct!',0 ;正确时的显示信息

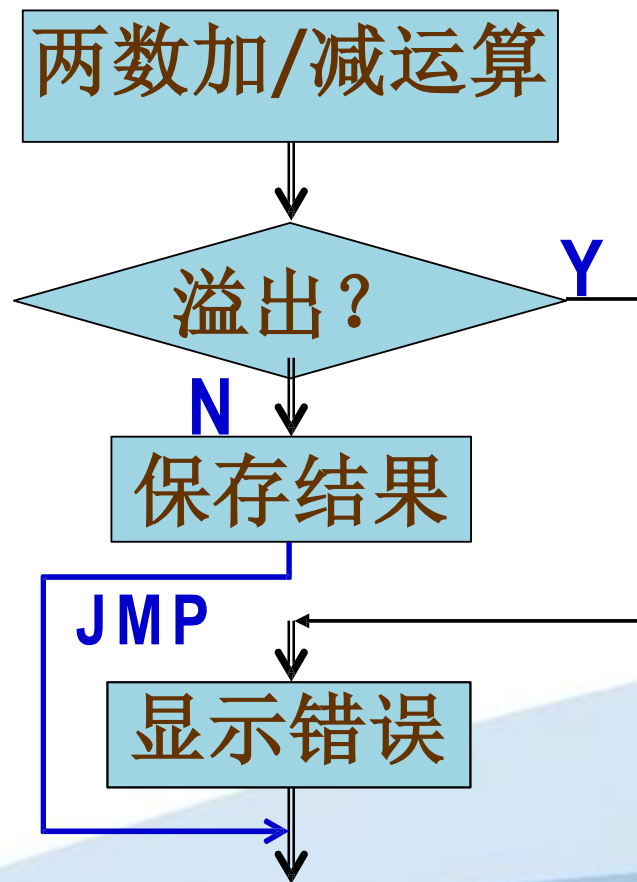
errmsg byte 'ERROR! Overflow!',0

;错误时的显示信息



有符号数运算溢出程序

```
mov eax,dvar1  
sub  eax,dvar2  
jo  error  
mov dvar3,eax  
mov eax,offset okmsg  
jmp disp  
error: mov eax,offset errmsg  
disp: call dispmsg
```



有符号数运算溢出程序

mov eax,dvar1

sub eax,dvar2

;求差

jo error

;有溢出，转移

mov dvar3,eax

;无溢出，保存差值

mov eax,offset okmsg

;显示 “Correct!”

jmp disp

error:

mov eax,offset errmsg

;显示 “ERROR!”

disp:

call dispmsg

双分支程序结构

